

FastAPI 기반 LLM 백엔드 API 개발과 스트리밍 실습

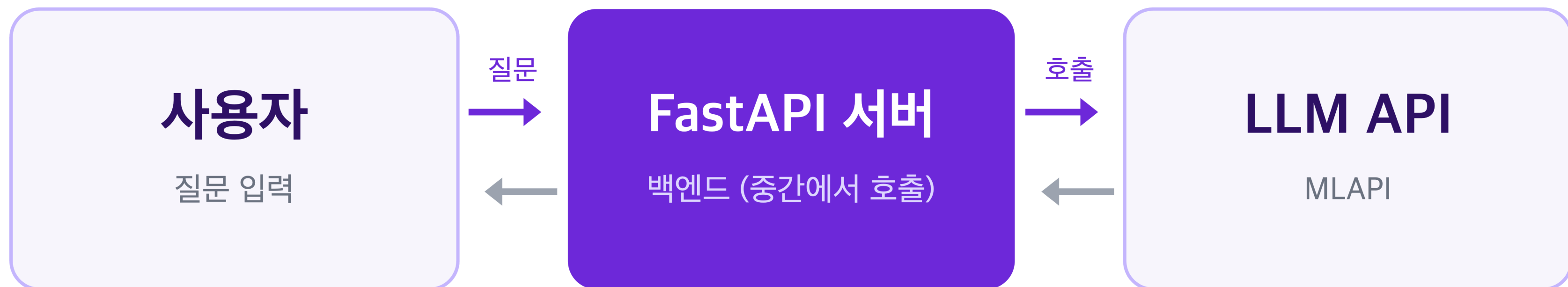
03 OpenAI API 연동 및 LLM 엔드포인트 구현

OpenAI / MLAPI 사용 환경 이해



LLM API를 백엔드에서 호출한다는 것

- LLM 기능은 보통 백엔드 서버 내부에서 외부 API 호출로 연결됨
- 사용자 질문 → FastAPI 서버 → LLM API 호출 → LLM 응답 수신 → 사용자에게 응답 반환
- 핵심은 사용자가 LLM API를 직접 호출하는 것이 아니라, 백엔드 서버가 중간에서 요청을 받아 LLM API를 호출한다는 점



사용자가 LLM API를 직접 호출하지 않고, 백엔드 서버가 중간에서 호출합니다



OpenAI API와 MLAPI의 위치

- OpenAI API는 LLM 호출의 표준 스펙 (Chat Completions)
- MLAPI는 OpenAI 호환 게이트웨이로, 같은 요청·응답 스펙을 그대로 따름
- 이 과정에서는 MLAPI를 통해 호출
 - FastAPI 서버 -> MLAPI -> OpenAI 호환 응답

OpenAI API

LLM 호출의 표준 스펙

Chat Completions 요청·응답
형식을 정의하는 기준

호환
↔

MLAPI

OpenAI 호환 게이트웨이

같은 요청·응답 스펙을
그대로 따르는 게이트웨이

Base URL만 MLAPI 주소로 지정하면 코드는 OpenAI API와 동일하게 다룰 수 있습니다



LLM API 호출에 필요한 정보

- LLM API를 호출하려면 보통 다음 정보 필요

1 API Key 인증 정보 — 누가 요청하는지

2 Base URL 요청을 보낼 주소 — 어디로 보낼지

3 Model 이름 어떤 모델을 사용할지 선택

4 사용자 입력 메시지 사용자의 질문 내용

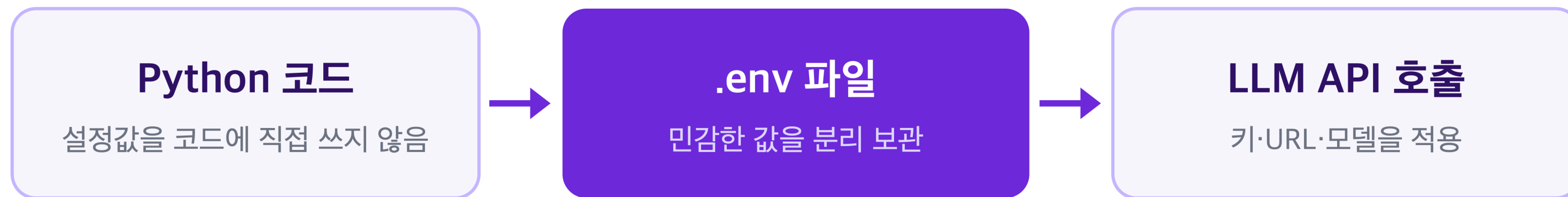
5 요청 옵션 응답 다양성 등 호출 옵션

- 이 과정에서는 요청을 보낼 Base URL은 MLAPI Base URL로 사용



.env로 설정값 관리하기

- API Key 같은 값은 코드에 직접 작성하지 않음



● ● ● .env

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxx
```

```
OPENAI_BASE_URL=https://api.example.com/v1
```

```
OPENAI_MODEL=gpt-4o-mini
```



LLM 요청의 기본 구조

- LLM API 요청은 보통 model, messages, temperature로 이루어짐
- messages는 대화 내용을 **역할별로 구성**하는 구조

model

사용할 모델 이름

messages

역할별 대화 내용 리스트

temperature

응답의 다양성 (0~2)

요청 messages

```
messages = [  
    {"role": "system", "content": "친절하게 답변하세요."},  
    {"role": "user", "content": "FastAPI가 뭐야?"}  
]
```




LLM 응답에서 사용할 정보

- LLM API 응답에는 여러 정보가 포함될 수 있지만, 백엔드 API에서는 보통 사용자에게 보여줄 **답변 텍스트를 추출**해 사용



● ● ● 응답에서 사용할 부분

LLM 원본 응답에서 답변이 있는 위치

```
response.choices[0].message.content
```

/chat API가 클라이언트에 내려주는 형태

```
{ "answer": "FastAPI는 Python 기반 API 프레임워크입니다." }
```

API Key 발급 및 기본 호출 테스트 (실습)



API Key 발급 및 기본 호출 테스트

실습 목표

- requests로 교육 환경의 MLAPI(생성형 AI API)에 직접 한 번의 호출을 보내본다
- 이번 실습이 끝나면 "**API 호출 한 번이 어떻게 생겼는지**" 감을 잡는 게 목표

실습 1. API Key 발급 및 기본 호출 테스트

실습 목표

이번 실습에서는 `requests`를 사용해 교육 환경에서 제공되는 생성형 AI API(MLAPI)에 한 번의 호출을 보내고, 정상 응답을 받는 과정을 익힙니다.

이번 실습에서 다루는 내용은 다음과 같습니다.

- MLAPI 접속 URL과 API Key 준비
- `.env` 파일로 필요한 설정값 분리
- 환경변수 또는 임시 입력으로 API Key 불러오기
- Chat Completions 형태의 `payload` 구성
- `requests.post()`로 기본 요청 보내기
- 응답 상태 코드와 응답 본문 확인
- 응답 JSON에서 답변 텍스트 추출하기



실습 흐름

1 라이브러리 불러오기

2 접속 정보 구조 살펴보기

3 환경변수 불러오기

4 기본 payload 구성하기

5 헤더 구성하기

6 requests로 기본 호출 실행하기

7 응답 JSON 구조 확인하기

8 답변 텍스트 추출하기

9 실패 신호 확인하기 (참고)



1. 라이브러리 불러오기

- HTTP 요청 전송, .env 환경변수 로드, API Key 안전 입력을 위한 라이브러리 불러오기

1. 라이브러리 불러오기

이번 실습에서 사용할 표준 라이브러리와 외부 패키지를 한 번에 불러옵니다.

```
import os
import json
import requests
from getpass import getpass
from dotenv import load_dotenv
```

Python



2. 접속 정보 구조 살펴보기

- 실습 포인트:
 - API 호출에 필요한 세 가지: BASE_URL / API_KEY / MODEL
 - 실제 호출 시 BASE_URL 뒤에 `/chat/completions`를 붙여 사용
 - 모델명은 `openai/gpt-4o-mini` 처럼 프로바이더 접두사 포함

2. 접속 정보 구조 살펴보기

이번 실습에서는 다음 형태의 요청 코드를 사용합니다.

```
response = requests.post(endpoint, headers=headers, json=payload)
```

API 호출에는 보통 다음 정보가 필요합니다.

MLAPI_BASE_URL → 요청을 보낼 API 베이스 주소 (끝이 `/v1`)
MLAPI_API_KEY → 인증에 사용할 API Key
MLAPI_MODEL → 사용할 모델 이름 (프로바이더 접두사 포함)

이번 실습은 `gpt-4o-mini` 기준 MLAPI 베이스 URL을 사용합니다.

`https://mlapi.run/40cc17ae-a89b-4f12-a7d6-13293180fc87/v1`

MLAPI는 OpenAI Chat Completions와 동일한 스펙을 따르므로 실제 호출 시에는 베이스 URL 뒤에 `/chat/completions`를 붙여 사용합니다. 모델명은 `openai/gpt-4o-mini` 처럼 프로바이더 접두사가 포함된 형태로 지정합니다.

아래 셀은 실제 비밀값이 아닌 예시 `.env` 파일을 만듭니다.

```
%%writefile .env.example
MLAPI_BASE_URL=https://mlapi.run/40cc17ae-a89b-4f12-a7d6-13293180fc87/v1
MLAPI_API_KEY=여기에_MLAPI_KEY를_입력하세요
MLAPI_MODEL=openai/gpt-4o-mini
```



3. 환경변수 불러오기

- 실습 포인트:
 - .env가 있으면 load_dotenv() → os.getenv()로 값 로드
 - Key가 비어있으면 getpass()로 셀 출력에 노출되지 않게 입력받기
 - 민감 정보를 코드에 하드코딩하지 않는 습관

3. 환경변수 불러오기

`.env` 파일이 있는 경우 값을 불러옵니다.

API Key가 환경변수에 없으면 노트북 실행 중 임시로 입력할 수 있습니다. 입력한 값은 코드 셀 출력에 표시되지 않습니다.

```
load_dotenv()

base_url = os.getenv("MLAPI_BASE_URL", "https://mlapi.run/40cc17ae-a89b-4f12-a7d6-13293180fc87/v1")
api_key = os.getenv("MLAPI_API_KEY")
model_name = os.getenv("MLAPI_MODEL", "openai/gpt-4o-mini")

if not api_key or api_key.startswith("여기에"):
    api_key = getpass("MLAPI API Key를 입력하세요: ")

print("MLAPI Base URL:", base_url)
print("모델명:", model_name)
print("API Key 입력 여부:", bool(api_key))
```

Python



4. 기본 payload 구성하기

- 실습 포인트:
 - Chat Completions payload의 핵심 필드: `model`, `messages`, `temperature`
 - `messages`는 `{"role": ..., "content": ...}` 객체의 리스트 (system / user)
 - `temperature`는 답변 다양성 (낮을수록 일관, 높을수록 자유로움)

4. 기본 payload 구성하기

MLAPI에 보낼 요청 Body를 구성합니다.

이번 실습에서는 Chat Completions 형태에 맞춰 `model`, `messages`, `temperature`를 사용합니다.

```
payload = {
    "model": model_name,
    "messages": [
        {"role": "system", "content": "너는 FastAPI와 백엔드 API를 쉽게 설명하는 교육 조교야."},
        {"role": "user", "content": "FastAPI를 한 문장으로 설명해줘."}
    ],
    "temperature": 0.2
}

print(json.dumps(payload, ensure_ascii=False, indent=2))
```

Python



5. 헤더 구성하기

- 실습 포인트:
 - MLAPI 인증 방식: **Authorization: Bearer** <API_KEY>
 - Content-Type: application/json 함께 지정
 - 출력 시에는 Key를 그대로 찍지 말고 **마스킹한 사본**으로 확인

5. 헤더 구성하기

MLAPI는 `Authorization: Bearer <API_KEY>` 헤더로 인증합니다.

Key 값 자체는 노출되지 않도록, 출력용으로는 마스킹한 사본을 만들어 확인합니다.

```
headers = {  
    "Accept": "application/json",  
    "Content-Type": "application/json",  
    "Authorization": f"Bearer {api_key}"  
}  
  
safe_headers = {**headers, "Authorization": "Bearer ***"}  
print(json.dumps(safe_headers, ensure_ascii=False, indent=2))
```

Python



6. requests로 기본 호출 실행하기

- 실습 포인트:
 - 엔드포인트 = `base_url.rstrip("/") + "/chat/completions"`
 - `requests.post(endpoint, json=payload, headers=headers, timeout=60)`
 - 호출 후 가장 먼저 확인할 것: `status_code` → `response.text`

6. requests로 기본 호출 실행하기

`requests.post()` 를 사용해 MLAPI에 요청을 보냅니다.

엔드포인트는 베이스 URL 뒤에 `/chat/completions` 를 붙여 만듭니다. 네트워크 상태, API Key, 모델명, payload 구조에 따라 오류가 발생할 수 있습니다.

```
endpoint = base_url.rstrip("/") + "/chat/completions"

try:
    response = requests.post(endpoint, json=payload, headers=headers, timeout=60)

    print("호출 완료")
    print("Status Code:", response.status_code)
    print("Response Text:")
    print(response.text[:1000])

except Exception as e:
    response = None
    print("호출 실패")
    print("오류 타입:", type(e).__name__)
    print("오류 메시지:", str(e)[:500])
```



7. 응답 JSON 구조 확인하기

- 실습 포인트:
 - response.json()으로 dict 변환
 - 먼저 최상위 Key부터 보고 → 필요한 부분만 들어간다
 - 응답을 잘 모르는 상태에서 바로 깊은 키로 들어가지 않기

7. 응답 JSON 구조 확인하기

응답이 JSON 형태라면 Python dict로 변환해 구조를 확인합니다.

```
if response is not None:
    try:
        response_json = response.json()

        print("응답 최상위 Key")
        print(list(response_json.keys()))

        print("\n응답 JSON 일부")
        print(json.dumps(response_json, ensure_ascii=False, indent=2)[:1500])

    except Exception as e:
        response_json = None
        print("JSON 변환 실패")
        print("오류 타입:", type(e).__name__)
        print("오류 메시지:", str(e)[:500])
else:
    response_json = None
    print("response가 없어 JSON 구조를 확인하지 않습니다.")
```




8. 답변 텍스트 추출하기

- 실습 포인트:
 - 답변 텍스트 위치: `choices[0].message.content`
 - 토큰 사용량 위치: `usage` (입력·출력 토큰)
 - 응답 구조를 확인한 뒤 필요한 값만 꺼낸다

8. 답변 텍스트 추출하기

OpenAI Chat Completions 형태의 응답이라면 답변 텍스트는 보통 다음 위치에 있습니다.

```
choices[0].message.content
```

MLAPI 응답 구조가 다를 수 있으므로, 먼저 실제 응답 JSON 구조를 확인한 뒤 필요한 값을 추출합니다.

```
if response_json is not None:
    try:
        answer = response_json["choices"][0]["message"]["content"].strip()
        usage = response_json.get("usage", {})

        print("LLM 답변")
        print("-" * 60)
        print(answer)
        print("-" * 60)
        print("사용 토큰:", usage)
    except Exception as e:
        print("답변 추출 실패")
        print("오류 타입:", type(e).__name__)
        print("오류 메시지:", str(e)[:500])
else:
    print("response_json이 없어 답변을 추출하지 않습니다.")
```

Python



9. 실패했을 때 자주 보는 신호 (참고)

- 실습 포인트:
 - 401 → API Key 누락/오류
 - 404 → 엔드포인트 경로 오류 (/chat/completions 누락 등)
 - 400 → 모델명 오류, 필수 필드 누락
 - 429 → 속도 제한, 잠시 후 재시도

9. 실패했을 때 자주 보는 신호 (참고)

Status	의미	가장 흔한 원인
401	인증 실패	MLAPI_API_KEY가 비었거나 잘못됨
404	경로 없음	엔드포인트에 /chat/completions가 빠짐
400	요청 오류	model 이름이 잘못되었거나 필수 필드 누락
429	속도 제한	잠시 후 재시도

위 표는 참고용입니다. 실제 에러 처리·재시도 로직은 다음 차시(LLM API 에러 처리 및 안정성)에서 다룹니다.



실습 정리

- MLAPI 베이스 URL과 API Key 구조 확인
- .env.example 구조를 살펴보고 환경변수로 설정값 분리
- Chat Completions 형태의 payload 구성
- requests.post()로 생성형 AI API 호출
- 응답 상태 코드와 응답 본문 확인
- 응답 JSON에서 답변 텍스트 추출

Python에서 OpenAI API 직접 호출 (실습)

실습 목표

- 실습 1에서 requests로 직접 보낸 호출을 공식 openai SDK 기반으로 다시 정리

실습 2. Python에서 OpenAI API 직접 호출

실습 목표

실습 1에서는 `requests`로 MLAPI에 직접 HTTP 요청을 보내봤습니다. 이번 실습에서는 공식 `openai` 파이썬 SDK를 사용해 같은 작업을 더 간결하게 수행하고, `messages` 구조와 `model` 파라미터, 응답 객체 구조를 단계별로 확인합니다.

이번 실습에서 다루는 내용은 다음과 같습니다.

- `openai` 클라이언트를 MLAPI에 연결하기 (`base_url`, `api_key`)
- `messages` 리스트로 system / user / assistant 역할 구성
- `model`, `temperature`, `max_tokens` 등 주요 파라미터 다루기
- `ChatCompletion` 응답 객체 구조 파헤치기 (`choices`, `message`, `usage`, `finish_reason`)
- 멀티턴 대화 흐름 만들어보기

실습 흐름

1 라이브러리 불러오기

2 클라이언트 만들기

3 가장 간단한 호출 (single-turn)

4 messages 구조 이해하기

5 주요 파라미터 다뤄보기

6 응답 객체 구조 확인하기

7 멀티턴 대화 만들어보기

TODO system 페르소나 · 정보 추출 함수 · 멀티턴 비교 · 챗봇 루프를 직접 구현

1. 라이브러리 불러오기

- SDK 호출, .env 환경변수 로드, API Key 안전 입력을 위한 라이브러리 불러오기

1. 라이브러리 불러오기

이번 실습에서 사용할 표준 라이브러리와 외부 패키지를 한 번에 불러옵니다.

```
import os
import json
from getpass import getpass
from dotenv import load_dotenv
from openai import OpenAI
```

Python

`openai` SDK는 OpenAI 공식 클라이언트이지만, `base_url` 만 바꾸면 OpenAI Chat Completions 스펙을 따르는 다른 서비스(MLAPI 등)에도 그대로 사용할 수 있습니다.

실습 1에서는 `requests` 로 `endpoint`, `headers`, `payload` 를 직접 조합했지만, SDK를 쓰면 그 과정을 클라이언트 객체 하나로 추상화할 수 있습니다.



2. 클라이언트 만들기

- 실습 포인트:
 - OpenAI(base_url=..., api_key=...) 한 줄로 접속·인증 정보가 클라이언트 객체 안에 캡슐화
 - **base_url만 바꾸면** OpenAI 스펙을 따르는 다른 서비스(MLAPI 등)에도 동일하게 사용
 - 이후 모든 호출은 client.chat.completions.create(...) 형태로 통일

2. 클라이언트 만들기

실습 1과 동일하게 .env 에서 접속 정보를 불러와 OpenAI 클라이언트를 만듭니다.

```
load_dotenv()

base_url = os.getenv("MLAPI_BASE_URL", "https://mlapi.run/40cc17ae-a89b-4f12-a7d6-13293180fc87/v1")
api_key = os.getenv("MLAPI_API_KEY")
model_name = os.getenv("MLAPI_MODEL", "openai/gpt-4o-mini")

if not api_key or api_key.startswith("여기예"):
    api_key = getpass("MLAPI API Key를 입력하세요: ")

client = OpenAI(base_url=base_url, api_key=api_key)

print("Base URL:", client.base_url)
print("모델명:", model_name)
```

Python

OpenAI(base_url=..., api_key=...) 한 줄로, 이후 모든 호출에 필요한 인증·접속 정보가 클라이언트 객체 안에 캡슐화됩니다. 앞으로 우리는 client.chat.completions.create(...) 형태로만 호출하면 됩니다.



3. 가장 간단한 호출 (single-turn)

- 실습 포인트:
 - user 메시지 하나만으로 한 번 호출하는 가장 단순한 형태
 - 실습 1의 7~8단계(endpoint 조립 → 호출 → JSON 파싱)가 **한 줄로 압축됨**
 - 답변 위치는 동일하게 response.choices[0].message.content

3. 가장 간단한 호출 (single-turn)

먼저 user 메시지 하나로 가장 단순한 호출을 보냅니다. 실습 1의 7~8단계에서 손으로 조립하던 작업이, 아래 한 호출로 동일하게 수행됩니다.

```
response = client.chat.completions.create(  
    model=model_name,  
    messages=[  
        {"role": "user", "content": "FastAPI를 한 문장으로 설명해줘."}  
    ],  
)  
  
print(response.choices[0].message.content)
```

Python

호출이 성공하면 모델의 답변이 출력됩니다. 만약 실패한다면 실습 1의 9번 표(Status Code별 원인)를 참고해 `MLAPI_BASE_URL`, `MLAPI_API_KEY`, `MLAPI_MODEL` 값을 점검합니다.

4. messages 구조 이해하기

- 실습 포인트:
 - messages는 대화 기록 리스트 — 각 메시지는 role + content
 - system: 모델의 성격·역할·규칙 / user: 사용자 입력 / assistant: 모델 이전 답변
 - 같은 user 질문이어도 **system을 바꾸면 톤·길이·관점이 크게 달라짐**

4. messages 구조 이해하기

Chat Completions API는 `messages` 라는 대화 기록 리스트를 입력으로 받습니다. 각 메시지는 `role` 과 `content` 로 구성됩니다.

role	설명
<code>system</code>	모델의 성격·역할·규칙을 정의하는 지침
<code>user</code>	사용자가 입력한 질문/요청
<code>assistant</code>	모델이 이전에 답변한 내용 (멀티턴에서 사용)

system 메시지를 추가해 모델에 역할과 답변 규칙을 부여해봅니다.

```
messages = [
    {"role": "system", "content": "너는 FastAPI와 백엔드 API를 쉽게 설명하는 교육 조교야. 답변은 3문장 이내로 짧게 해."},
    {"role": "user", "content": "FastAPI가 Flask와 비교해 가지는 장점이 뭐야?"}
]

response = client.chat.completions.create(
    model=model_name,
    messages=messages,
)

print(response.choices[0].message.content)
```

Python

system 메시지를 통해 답변의 톤·길이·관점을 제어할 수 있다는 점이 핵심입니다. 같은 user 질문이라도 system이 어떻게 쓰여 있느냐에 따라 답변 스타일이 크게 달라집니다.

5. 주요 파라미터 다뤄보기

- 실습 포인트:
 - temperature: 답변의 무작위성 (낮을수록 일관, 높을수록 창의적·변동성↑)
 - max_tokens: 응답 최대 토큰 수 — 너무 작게 주면 답변이 중간에 잘림
 - 응답이 잘리면 **finish_reason**이 "length", 정상 종료는 "stop"
 - 동일 질문을 값만 바꿔 비교해보면서 감 잡기

5. 주요 파라미터 다뤄보기

자주 사용하는 파라미터는 다음과 같습니다.

파라미터	의미	일반적 값
model	사용할 모델 이름	openai/gpt-4o-mini 등
temperature	답변의 무작위성 (낮을수록 일관)	0.0 ~ 1.0
max_tokens	응답의 최대 토큰 수	64 ~ 1024
top_p	후보 토큰 확률 누적 컷오프	0.0 ~ 1.0

가장 자주 만지게 되는 temperature 부터 살펴봅니다. 동일한 질문을 temperature 값만 바꿔 호출해 답변이 어떻게 달라지는지 비교해봅니다.

```
question = "FastAPI로 만든 챗봇의 매력 포인트를 한 줄로 표현해줘."

for temp in [0.0, 0.7, 1.2]:
    response = client.chat.completions.create(
        model=model_name,
        messages=[{"role": "user", "content": question}],
        temperature=temp,
        max_tokens=80,
    )
    print(f"[temperature={temp}]")
    print(response.choices[0].message.content)
    print("-" * 60)
```

Python

낮은 temperature 는 정형화된 답변, 높은 temperature 는 창의적이지만 변동성이 큰 답변을 만들어냅니다.

💡 temperature=0 이라고 해서 항상 100% 동일한 결과가 나오지는 않을 수 있습니다. 다만 비교적 안정적인 답변을 얻을 수 있습니다.

이번에는 max_tokens 가 어떻게 작동하는지 확인해봅니다. 응답이 잘릴 수 있는 값으로 일부러 작게 줍니다.

```
long_question = "FastAPI를 처음 배우는 사람이 알아야 할 핵심 개념을 5가지 정도로 정리해줘."

for limit in [16, 256]:
    response = client.chat.completions.create(
        model=model_name,
        messages=[{"role": "user", "content": long_question}],
        max_tokens=limit,
    )
    print(f"[max_tokens={limit}] finish_reason={response.choices[0].finish_reason}")
    print(response.choices[0].message.content)
    print("-" * 60)
```

Python

6. 응답 객체 구조 확인하기

- 실습 포인트:
 - 응답은 dict가 아닌 **ChatCompletion** 객체 → 점(.) 접근
 - 주요 필드:
 - `choices[0].message.content` → 답변 텍스트
 - `choices[0].finish_reason` → 종료 사유 (stop / length / content_filter 등)
 - `usage` → 입력·출력 토큰 수
 - 구조 자체는 실습 1의 JSON과 동일 (`response.model_dump()`로 dict 변환 가능)

6. 응답 객체 구조 확인하기

실습 1에서는 응답을 dict(JSON)로 다뤘지만, SDK를 사용하면 `ChatCompletion` 객체로 돌려받습니다. 점(.) 접근으로 필드를 꺼낼 수 있습니다.

```
response = client.chat.completions.create(  
    model=model_name,  
    messages=[  
        {"role": "system", "content": "너는 친절한 백엔드 멘토야."},  
        {"role": "user", "content": "REST API의 핵심 원칙 3가지를 알려줘."}  
    ],  
    temperature=0.3,  
)  
  
print("타입:", type(response).__name__)  
print("id:", response.id)  
print("모델:", response.model)  
print("choices 개수:", len(response.choices))  
print()  
  
choice = response.choices[0]  
print("finish_reason:", choice.finish_reason)  
print("message.role:", choice.message.role)  
print("message.content:")  
print(choice.message.content)  
print()  
  
print("usage:", response.usage)
```

Python

각 필드의 의미는 다음과 같습니다.

- `choices`: 생성된 응답 후보 리스트 (보통 1개)
- `choices[0].message.content`: 실제 답변 텍스트
- `choices[0].finish_reason`: 응답이 끝난 이유 (stop, length, content_filter 등)
- `usage`: 입력/출력에 사용된 토큰 수 (prompt_tokens, completion_tokens, total_tokens)

객체 전체 구조가 궁금하다면 dict로 변환해 한 번 들여다봅시다.

```
print(json.dumps(response.model_dump(), ensure_ascii=False, indent=2)[:1500])
```

Python

이 구조는 실습 1에서 본 JSON과 완전히 동일합니다. SDK는 그 위에 객체 형태의 편의 인터페이스를 얹어 준 것뿐입니다.

7. 멀티턴 대화 만들어보기

- 실습 포인트:
 - LLM API는 **stateless** — 서버는 이전 대화를 기억하지 않음
 - 맥락 유지하려면 클라이언트가 messages에 이전 대화를 누적해서 매번 같이 보내야 함
 - "방금 말한 것 중 첫 번째..." 같은 후속 질문이 자연스럽게 동작하는 이유 = 이전 user·assistant 메시지가 함께 전송되기 때문

7. 멀티턴 대화 만들어보기

LLM API는 기본적으로 **stateless** 입니다. 모델 쪽에 대화 기록이 저장되지 않으므로, 맥락을 유지하려면 클라이언트가 **messages** 리스트에 이전 대화를 누적해서 매번 함께 보내야 합니다.

```
conversation = [
    {"role": "system", "content": "너는 FastAPI 학습을 돕는 교육 조교야."},
]

def ask(user_text: str):
    conversation.append({"role": "user", "content": user_text})
    response = client.chat.completions.create(
        model=model_name,
        messages=conversation,
        temperature=0.3,
    )
    answer = response.choices[0].message.content
    conversation.append({"role": "assistant", "content": answer})
    return answer

print("Q1:", "FastAPI의 핵심 컴포넌트 3가지만 알려줘.")
print("A1:", ask("FastAPI의 핵심 컴포넌트 3가지만 알려줘."))
print("-" * 60)

print("Q2:", "방금 말한 것 중 첫 번째를 좀 더 자세히 설명해줘.")
print("A2:", ask("방금 말한 것 중 첫 번째를 좀 더 자세히 설명해줘."))
print("-" * 60)

print("최종 messages 길이:", len(conversation))
```

Python

두 번째 질문("방금 말한 것 중 첫 번째...")에 모델이 자연스럽게 답하는 이유는, **messages**에 첫 질문과 답변이 함께 누적되어 전달되었기 때문입니다. 만약 두 번째 질문을 단일 user 메시지로만 보냈다면 모델은 "무엇의 첫 번째인지" 알 수 없어 일반적인 답변을 했을 것입니다.



TODO

- TODO 1. system 메시지로 페르소나 바꾸기
- TODO 2. 응답 객체에서 정보 추출 함수 만들기
- TODO 3. 멀티턴 vs 단일턴 직접 비교
- TODO 4. 간단한 챗봇 루프 만들기

실습 정리

- openai SDK로 MLAPI에 연결 (OpenAI(base_url=..., api_key=...))
- messages의 system / user / assistant 역할 구조 이해
- temperature, max_tokens 등 주요 파라미터의 효과 비교
- ChatCompletion 응답 객체의 choices · message · usage · finish_reason 확인
- messages에 대화 기록을 누적해 멀티턴 대화 구성

FastAPI와 OpenAI API 연동 (실습)



실습 목표

- 노트북에서 직접 호출하던 LLM을 **FastAPI 백엔드 뒤로 숨기는** 가장 단순한 형태의 /chat API 만들기
- 이번 실습은 노트북 안에서 끝나지 않고 main.py, .env, .env.example, .gitignore 파일을 만들어 사용

실습 3. FastAPI와 OpenAI API 연동

실습 목표

실습 2에서는 노트북 안에서 직접 `openai` SDK로 LLM을 호출했습니다. 하지만 실제 서비스에서는 LLM을 **백엔드 API 뒤에 숨기고** 클라이언트는 백엔드 API만 호출하게 만듭니다. 이번 실습에서는 FastAPI 엔드포인트 내부에서 LLM을 호출하는 가장 단순한 형태의 `/chat` API를 만들어 봅니다.

이번 실습에서 다루는 내용은 다음과 같습니다.

- FastAPI 앱을 만들고 uvicorn으로 실행하는 흐름 복습
- `.env` 로 API Key/모델/엔드포인트 분리 관리
- 모듈 레벨에서 OpenAI 클라이언트를 한 번만 생성해 재사용
- Pydantic 모델로 request/response 스키마를 최소한으로 정의
- `POST /chat` 엔드포인트 안에서 LLM 호출
- Swagger UI와 노트북에서 직접 API 테스트
- API Key 관련 보안 유의사항

입출력 스키마 설계, 에러 처리, /chat API 통합 마무리는 이어지는 차시에서 더 정교하게 다룹니다. 이번 실습은 "FastAPI ↔ LLM"이 일 단 한 번 연결되는 흐름을 확실히 잡는 게 목표입니다.

이번 실습은 노트북 한 파일에서 끝나지 않습니다. 다음과 같은 파일들을 만들어 사용합니다.

실습폴더/

— main.py	# FastAPI 앱
— .env	# 실제 API Key (절대 공유 금지)
— .env.example	# 동료에게 공유 가능한 예시 파일
— .gitignore	# .env가 깃에 올라가지 않도록 막는 파일



실습 흐름

1 왜 FastAPI 뒤에 LLM을 둘까?

2 최소 FastAPI 앱 만들기

3 .env로 설정값 분리하기

4 FastAPI 앱에서 .env 불러오기

5 /chat 엔드포인트 만들기

6 동작 확인하기

7 보안 유의사항 정리

TODO

system_prompt · temperature 옵션 · 토큰 사용량 반환 등을 직접 구현



1. 왜 FastAPI 뒤에 LLM을 둘까?

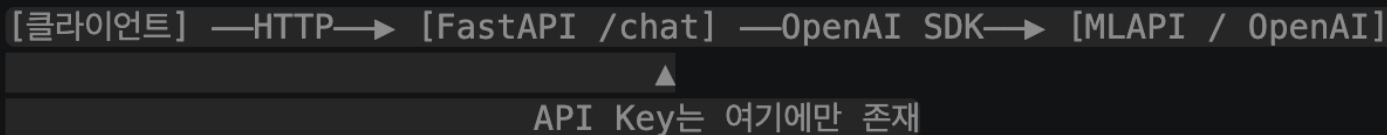
- 실습 포인트:
 - 클라이언트가 LLM을 직접 호출하면 생기는 문제
 - API Key 노출 / 사용량·비용 통제 불가 / 비즈니스 로직 분리 불가 / 다양한 클라이언트 대응 곤란
- 실무 구조:
 - [클라이언트] → [FastAPI /chat] → [MLAPI / OpenAI]
- API Key는 **FastAPI 서버에만 존재** — 클라이언트로 절대 내려보내지 않음

1. 왜 FastAPI 뒤에 LLM을 둘까?

실습 2까지는 노트북에서 곧바로 MLAPI를 호출했습니다. 학습 중에는 편하지만 실제 서비스에서는 다음과 같은 문제가 생깁니다.

문제	설명
API Key 노출	브라우저/모바일 클라이언트에서 직접 호출하면 Key가 사용자에게 그대로 노출됩니다.
사용량 통제 불가	누가 얼마나 썼는지 백엔드가 모르고, 악의적 사용자가 비용을 폭증시킬 수 있습니다.
비즈니스 로직 분리 불가	system 프롬프트, 모델 선택, 후처리 규칙 등을 클라이언트가 마음대로 바꿀 수 있게 됩니다.
다양한 클라이언트 대응	웹/앱/배치 작업 등이 같은 규칙으로 LLM을 쓰려면 공통 진입점이 필요합니다.

그래서 실무에서는 다음과 같은 구조를 사용합니다.



이번 실습에서 만들 것이 바로 가운데의 **FastAPI /chat** 입니다.



2. 최소 FastAPI 앱 만들기

- 실습 포인트:
 - LLM 없이 먼저 /health 하나로 동작하는 FastAPI 앱부터 띄워보기
 - 서버 실행: uvicorn main:app --reload --port 8000
 - -- reload 옵션으로 코드 변경 시 자동 재시작
 - 브라우저로 <http://127.0.0.1:8000/docs> 접속 시 Swagger UI 자동 제공

2. 최소 FastAPI 앱 만들기

먼저 LLM 없이, 동작하는 FastAPI 앱을 한 번 띄워봅니다. 이전 일차에서 다뤘지만 워밍업 겸 복습합니다.

```
%%writefile main.py
from fastapi import FastAPI

app = FastAPI(title="LLM Chat Backend")

@app.get("/health")
def health():
    return {"status": "ok"}
```

Python

이제 터미널을 새로 열어 실습 폴더로 이동한 다음 아래 명령으로 서버를 띄웁니다.

```
uvicorn main:app --reload --port 8000
```

--reload 는 코드를 수정할 때마다 서버를 자동 재시작해주는 옵션입니다. 학습용으로 매우 유용합니다.

서버가 뜨면 노트북으로 돌아와 동작을 확인합니다.

```
import requests

response = requests.get("http://127.0.0.1:8000/health")
print(response.status_code)
print(response.json())
```

Python

{"status": "ok"} 가 출력되면 성공입니다.

💡 터미널을 띄울 수 없는 환경이라면? 아래 셀처럼 노트북 안에서 백그라운드로 서버를 띄울 수도 있습니다. 단, 터미널에서 실행하는 쪽이 로그 확인이 훨씬 쉬우니 가능한 경우 터미널을 권장합니다.

```
# (옵션) 백그라운드 폴백: 터미널을 쓸 수 없을 때만 사용
import subprocess, time

server = subprocess.Popen(
    ["uvicorn", "main:app", "--reload", "--port", "8000"],
    stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True,
)

time.sleep(2)
print("uvicorn 실행 중, PID:", server.pid)
# 종료하려면: server.terminate()
```

Python



3. .env로 설정값 분리하기

- 실습 포인트:
 - .env.example — 동료에게 공유 가능한 예시 파일 (실제 값 없음)
 - .env — 실제 사용할 Key가 들어가는 파일, 절대 공유 금지
 - .gitignore에 .env 포함 → 깃 푸시로 인한 Key 유출 차단
 - 실수로 한 번이라도 커밋했다면 그 Key는 노출된 것, 즉시 재발급

3. .env 로 설정값 분리하기

실습 1-2에서 본 것처럼 API Key, 모델명, base URL은 코드에 직접 적지 않습니다. 대신 `.env` 파일에 분리해두고 코드에서는 환경변수로 읽어옵니다.

먼저 동료에게 공유 가능한 예시 파일을 만듭니다.

```
%writefile .env.example
MLAPI_BASE_URL=https://mlapi.run/40cc17ae-a89b-4f12-a7d6-13293180fc87/v1
MLAPI_API_KEY=여기에 MLAPI_KEY를 입력하세요
MLAPI_MODEL=openai/gpt-4o-mini
```

Python

그리고 실제로 사용할 `.env` 파일을 만듭니다. 아래 셀의 `MLAPI_API_KEY=` 뒤에 본인의 Key를 채워 넣은 뒤 실행하세요.

```
%%writefile .env  
MLAPI_BASE_URL=https://mlapi.run/40cc17ae-a89b-4f12-a7d6-13293180fc87/v1  
MLAPI_API_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpYXQiOjE3Nzg3NTQzM0AsIm5iZiI6MTc3ODc1NDM4MCwiZXhwIjoxNA.  
MLAPI_MODEL=openai/gpt-4o-mini
```

Python

마지막으로 `.env`가 `Git`에 올라가지 않도록 `.gitignore`도 만들어 둡니다. 이걸 단순한 습관이 아니라 보안 사고의 1순위 원인을 막는 핵심 안전장치입니다.

```
%writefile .gitignore
.env
__pycache__/
*.pyc
.ipynb_checkpoints/
```

Python

⚠️ **실수로라도 .env를 커밋해 GitHub에 푸시한 적이 있다면, 그 Key는 이미 노출된 것으로 보고 즉시 폐기·재발급해야 합니다.** 커밋 히스토리에서 지우는 것만으로는 부족합니다.



4. FastAPI 앱에서 .env 불러오기

- 실습 포인트:
 - load_dotenv() → os.getenv()로
BASE_URL · API_KEY · MODEL_NAME 읽기
 - OpenAI 클라이언트는 **모듈 레벨에서 한 번만 생성**해
모든 요청에서 재사용 (요청마다 새로 만들면 성능 손해)
 - API_KEY가 비어있으면 앱 기동 시점에 바로 RuntimeError로 멈춤
 - print(API_KEY) 같은 디버깅 코드 금물**
 - 굳이 찍어야 한다면 API_KEY[:4] + "..."로 마스킹

4. FastAPI 앱에서 .env 불러오기

이제 main.py를 확장해 .env를 로드하고, OpenAI 클라이언트를 모듈 레벨에서 한 번만 생성합니다. 요청이 들어올 때마다 클라이언트를 새로 만들면 매번 커넥션이 새로 맺어지므로 성능에 손해입니다.

```
%%writefile main.py
import os
from dotenv import load_dotenv
from fastapi import FastAPI
from openai import OpenAI

load_dotenv() # .env 파일을 환경변수로 로드

BASE_URL = os.getenv("MLAPI_BASE_URL")
API_KEY = os.getenv("MLAPI_API_KEY")
MODEL_NAME = os.getenv("MLAPI_MODEL", "openai/gpt-4o-mini")

if not API_KEY:
    raise RuntimeError("MLAPI_API_KEY가 설정되어 있지 않습니다. .env 파일을 확인하세요.")

# 모듈 레벨에서 한 번만 만들고 모든 요청에서 재사용한다.
client = OpenAI(base_url=BASE_URL, api_key=API_KEY)

app = FastAPI(title="LLM Chat Backend")

@app.get("/health")
def health():
    return {"status": "ok", "model": MODEL_NAME}
```

Python

--reload 옵션 덕분에 터미널의 uvicorn은 파일 변경을 감지해 자동으로 재시작합니다. 노트북에서 다시 호출해 모델명이 응답에 나오는지 확인합니다.

```
response = requests.get("http://127.0.0.1:8000/health")
print(response.json())
```

Python

💡 print(API_KEY) 같은 코드는 **절대로 작성하지 마세요**. 디버깅하다가 한 줄 찍어둔 게 그대로 로그 콘솔에 남는 사고가 의외로 자주 발생합니다. 값 자체를 출력해야 한다면 print(API_KEY[:4] + "...")처럼 일부만 마스킹합니다.



5. /chat 엔드포인트 만들기

- 실습 포인트:
 - 입출력은 Pydantic 모델로 최소한만 정의:
 - ChatRequest{message} / ChatResponse{answer}
 - response_model=ChatResponse 지정
→ 자동 검증·직렬화, 의도치 않은 필드 노출도 방지
 - system 메시지는 서버 코드에 고정
→ 클라이언트가 페르소나를 마음대로 못 바꾸게
 - 에러 처리는 다음 실습에서 진행

5. /chat 엔드포인트 만들기

이제 본격적으로 LLM을 호출하는 엔드포인트를 추가합니다. 입출력은 **Pydantic** 모델로 최소한만 정의합니다. (이 입출력 구조를 더 정교하게 만드는 작업은 다음 차시에서 다룹니다.)

```
%%writefile main.py
import os
from dotenv import load_dotenv
from fastapi import FastAPI
from pydantic import BaseModel
from openai import OpenAI

load_dotenv()

BASE_URL = os.getenv("MLAPI_BASE_URL")
API_KEY = os.getenv("MLAPI_API_KEY")
MODEL_NAME = os.getenv("MLAPI_MODEL", "openai/gpt-4o-mini")

if not API_KEY:
    raise RuntimeError("MLAPI_API_KEY가 설정되어 있지 않습니다. .env 파일을 확인하세요.")

client = OpenAI(base_url=BASE_URL, api_key=API_KEY)

app = FastAPI(title="LLM Chat Backend")

# --- 입출력 모델 ---
class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    answer: str

# --- 라우트 ---
@app.get("/health")
def health():
    return {"status": "ok", "model": MODEL_NAME}

@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest):
    completion = client.chat.completions.create(
        model=MODEL_NAME,
        messages=[
            {"role": "system", "content": "너는 FastAPI 학습을 돕는 친절한 백엔드 조교야."},
            {"role": "user", "content": req.message},
        ],
        temperature=0.3,
    )
    answer = completion.choices[0].message.content
    return ChatResponse(answer=answer)
```



6. 동작 확인하기

- 실습 포인트:
 - 세 가지 테스트 경로
 - Swagger UI: /docs -> POST /chat -> Try it out -> Execute
 - 노트북: requests.post(".../chat", json=payload)
 - curl: 터미널에서 -X POST ... -d '{"message": "..."}'
 - 잘못된 요청(message 누락 등)은 Pydantic이 자동으로 422로 막아줌

6. 동작 확인하기

세 가지 방법으로 확인할 수 있습니다.

(1) Swagger UI — 브라우저에서 `http://127.0.0.1:8000/docs` → POST /chat → Try it out → request body 입력 → Execute.

(2) 노트북에서 requests로 호출

앞 셀에서 main.py에 /chat을 추가해 다시 저장했으므로, 터미널의 --reload가 서버를 이미 자동 재시작했습니다. 별도 재시작 없이 바로 아래 셀을 실행하면 됩니다.

```
payload = {"message": "FastAPI에서 Pydantic이 하는 역할을 한 문장으로 알려줘."}

response = requests.post("http://127.0.0.1:8000/chat", json=payload, timeout=60)
print("Status:", response.status_code)
print("Response JSON:", response.json())
```

Python

(3) curl — 터미널에서:

```
curl -X POST http://127.0.0.1:8000/chat \
-H "Content-Type: application/json" \
-d '{"message": "FastAPI에서 Pydantic이 하는 역할을 한 문장으로 알려줘."}'
```

세 경우 모두 {"answer": "..."} 형태의 JSON이 돌아오면 성공입니다.

```
# 잘못된 요청을 보내면 어떻게 되는지도 한 번 확인해봅시다.
bad = requests.post("http://127.0.0.1:8000/chat", json={"msg": "오타!"})
print("Status:", bad.status_code)
print(bad.json())
```

Python

message 필드가 빠졌으므로 FastAPI가 422 Unprocessable Entity 와 함께 어디가 문제인지 친절하게 알려줍니다. 우리가 try/except를 쓰지 않았는데도 Pydantic 검증이 자동으로 막아준다는 점을 확인할 수 있습니다.



TODO

- TODO 1. /chat에 system_prompt를 선택적으로 받기
- TODO 2. /chat에 temperature를 선택 파라미터로 추가하기
- TODO 3. 응답에 토큰 사용량까지 함께 반환하기
- TODO 4. /health에 마지막 호출 시각 노출하기



실습 정리

- FastAPI 앱을 만들어 uvicorn으로 실행
- .env / .env.example / .gitignore로 설정값·비밀값 분리 관리
- OpenAI 클라이언트를 모듈 레벨에서 한 번만 생성해 재사용
- ChatRequest / ChatResponse Pydantic 모델로 입출력 스키마 정의
- POST /chat에서 LLM 호출 후 응답 반환
- Swagger UI · requests · curl 세 가지 방법으로 API 테스트

LLM 엔드포인트 입력·출력 구조 설계



왜 구조 설계가 필요한가

- 실습 3에서 /chat 엔드포인트를 Request/Response 모델로 직접 만들어봄
- 이번에는 그 입출력 구조를 "왜 그렇게 설계했는지" 관점에서 정리
- LLM 호출 코드만 있어도 동작은 함. 하지만 API로 외부에 열려면 **입력 형태와 출력 형태를 약속**해야 함

약속이 없으면

입력·출력 형태가 정해져 있지 않을 때 생기는 문제

클라이언트마다 다른 형태로 보냄



서버에서 매번 분기 처리

응답 구조가 그때그때 달라짐



프론트엔드에서 파싱이 깨짐

잘못된 입력이 그대로 LLM까지 전달



불필요한 비용·오류 증가

결론

엔드포인트마다 입력 모델 / 출력 모델을 먼저 설계합니다



/chat API 기본 흐름

- Request Model → /chat Endpoint → LLM API 호출 → Response Model
 - Request Model: 사용자가 보낸 입력 검증
 - /chat Endpoint: 요청을 받아 처리하는 진입점
 - LLM API 호출: MLAPI로 실제 요청 전송
 - Response Model: LLM 응답을 정리해서 클라이언트에 반환



사용자 입력을 검증하고 LLM을 호출해, 정리된 응답을 돌려주는 흐름



Request Model

- 역할: 클라이언트가 보낸 JSON을 검증된 파이썬 객체로 변환
- /chat에서 최소한으로 받아야 하는 것:

message

사용자 질문 (문자열)

필수

- 자주 같이 받는 것:

model

사용할 모델 이름

선택

temperature

생성 다양성 (0~2)

선택

- 설계 포인트:

설계 포인트

필수·선택을 명확히 구분 · 값 범위·타입 제약은 모델에 위임 · 옵션을 한 번에 너무 많이 노출하지 않기



Response Model

- 역할: LLM이 돌려준 원본 응답에서 클라이언트가 **필요한 부분만 정리**해서 내려줌
- /chat에서 최소한으로 내려줄 것

answer

LLM이 생성한 답변 텍스트

필수

- 자주 같이 내려주는 것

model

어떤 모델이 응답했는지

선택

usage

토큰 사용량

선택

- 그대로 내려주면 안 되는 이유

원본을 그대로 내리면 안 되는 이유

원본 응답 구조가 복잡함 · 구조가 바뀌면 클라이언트도 깨짐 · 불필요한 정보까지 외부 노출

LLM API 에러 처리 및 안정성 이해



왜 오류 처리가 필요한가?

- 실습 3에서는 /chat을 만들되 에러 처리는 일부러 넣지 않음
- 이번 섹션에서 그 에러 처리를 다루고, 다음 실습에서 직접 구현
- LLM API는 **외부 서비스**라 우리 서버 마음대로 통제 불가

외부 API

LLM API는 외부 서비스 — 우리 서버가 마음대로 통제할 수 없습니다

발생 가능한 상황

- 네트워크 일시 단절
- API 키 만료 · 권한 오류
- 요청 한도(Rate limit) 초과
- 응답 지연(Timeout)
- 모델 이름 오타 · 잘못된 파라미터

오류 처리를 안 하면

FastAPI가 그대로 500 에러를 던짐

클라이언트는 원인을 알 수 없음

서비스 전체가 불안정해 보임



왜 오류 처리가 필요한가?

- 발생 가능한 상황:
 - 네트워크 일시 단절
 - API 키 만료 / 권한 오류
 - 요청 한도(Rate limit) 초과
 - 응답 지연(Timeout)
 - 모델 이름 오타, 잘못된 파라미터
- 오류 처리를 안 하면:
 - FastAPI가 그대로 500 에러를 던짐
 - 클라이언트는 원인을 알 수 없음
 - 서비스 전체가 불안정해 보임



자주 만나는 오류 유형

- 인증 오류 (401):
 - API 키가 잘못되었거나 만료됨
- 요청 한도 초과 (429):
 - 짧은 시간에 너무 많이 호출
- 잘못된 요청 (400):
 - 모델 이름·파라미터·메시지 형식 오류
- 타임아웃:
 - 응답이 정해진 시간 안에 오지 않음
- 서버 오류 (5xx):
 - LLM 제공자 쪽 일시 장애

코드	유형	원인
401	인증 오류	API 키가 잘못되었거나 만료됨
429	요청 한도 초과	짧은 시간에 너무 많이 호출
400	잘못된 요청	모델명·파라미터·메시지 형식 오류
Timeout	타임아웃	응답이 정해진 시간 안에 오지 않음
5xx	서버 오류	LLM 제공자 쪽 일시 장애



try/except로 감싸는 위치

- 감싸는 위치: **LLM API를 호출하는 그 줄**
- 좋은 예:

● ● ● 오류 처리 – 좋은 예

```
try:  
    response = client.chat.completions.create(...)  
except Exception as e:  
    raise HTTPException(status_code=502, detail="LLM 호출 실패")
```

- 외부 호출(LLM API) 한 줄만 try로 감싼다 — 엔드포인트 전체를 감싸지 않는다
- 잡은 오류는 HTTPException으로 바꿔서 내려주고, 원본 예외 메시지는 노출하지 않는다



상태 코드를 의미 있게 내려주기

- 모든 실패를 500으로 내려주면 클라이언트가 원인 구분 불가

사용자 입력 문제



400

인증 문제



401

한도 초과



429

LLM 제공자 장애 · 타임아웃



502 / 504

detail에는 원인 카테고리만 표시 — 내부 스택트레이스·키 값은 절대 노출하지 않습니다



타임아웃과 재시도

타임아웃

timeout

- 1 LLM 응답이 너무 오래 걸리면
클라이언트도 서버도 자원이 묶임
- 2 호출할 때 timeout 값을
명시적으로 설정해 둬

재시도

retry

- 1 일시적인 네트워크·5xx 오류는
한두 번 재시도로 성공하기도 함
- 2 무한 재시도는 금지
횟수 제한 + 간격(backoff) 필요

두 가지 모두 '한계'를 정하는 장치 무한정 기다리지도, 무한정 재시도하지도 않습니다

/chat API 통합 구현 (실습)



실습 목표

- 지금까지 따로 다뤘던 것들을 하나의 /chat 엔드포인트로 통합
 - FastAPI 앱 + LLM 호출
 - Request Model / Response Model 적용
 - 기본 오류 처리 (try/except → HTTPException)
 - /docs로 정상 / 실패 케이스 테스트
- 이번 실습이 끝나면 외부에 열어도 어색하지 않은 형태의 /chat API 완성

실습 4 - /chat API 통합 구현

실습 목표

지금까지 따로 다뤘던 것들을 하나의 /chat 엔드포인트로 통합합니다.

- FastAPI 앱 + LLM 호출
- Request Model / Response Model 적용
- 기본 오류 처리 (try/except → HTTPException)
- /docs 로 정상 / 실패 케이스 테스트

이번 실습이 끝나면 외부에 열어도 어색하지 않은 형태의 /chat API가 완성됩니다.



실습 흐름

1 라이브러리 불러오기

2 LLM 호출 함수 정리

3 Request / Response Model 정의

4 /chat 엔드포인트 구현

5 오류 처리 붙이기

6 서버 실행 및 /docs 테스트

7 실패 케이스 살펴보기

TODO

Request·Response Model 확장 · 입력 검증 강화 · 오류 처리 보강을 직접 구현



1. 라이브러리 불러오기

- FastAPI 앱 · 노트북 내 서버 실행, SDK 호출, .env 로드, API Key 안전 입력을 위한 라이브러리 불러오기
- 실습은 노트북 한 파일 안에서 FastAPI 앱을 띄우고 같은 노트북에서 /docs로 테스트

1. 라이브러리 불러오기

```
import os
from getpass import getpass
from dotenv import load_dotenv
from openai import OpenAI
```

Python



2. LLM 호출 함수 정리

- 실습 포인트:
 - .env → os.getenv()로 BASE_URL · API_KEY · MODEL 읽기, 비어있으면 getpass()로 폴백
 - OpenAI(base_url=..., api_key=...) 클라이언트 생성 후 **call_llm()** 함수 한 곳에서 호출 로직 집중 관리
 - 함수 입력은 message / temperature, 출력은 응답 객체 전체 (필요한 값은 호출하는 쪽에서 꺼냄)

2. LLM 호출 함수 정리

이전 실습에서 만든 LLM 호출 함수를 다시 정리합니다.

- API Key, Base URL, 모델명은 .env 파일에서 먼저 읽고, 값이 없으면 입력 프롬프트로 직접 받습니다.
- LLM 클라이언트는 OpenAI 또는 교육 환경에서 제공되는 MLAPI 접속 정보를 사용합니다.

```
load_dotenv()

base_url = os.getenv("MLAPI_BASE_URL", "https://mlapi.run/40cc17ae-a89b-4f12-a7d6-13293180fc87/v1")
api_key = os.getenv("MLAPI_API_KEY")
model_name = os.getenv("MLAPI_MODEL", "openai/gpt-4o-mini")

if not api_key or api_key.startswith("여기에"):
    api_key = getpass("MLAPI API Key를 입력하세요: ")

client = OpenAI(base_url=base_url, api_key=api_key)

print("Base URL:", client.base_url)
print("모델명:", model_name)
```

Python

이제 LLM 호출 함수를 정의합니다.

- 입력: 사용자 메시지(문자열), temperature(선택)
- 출력: LLM 응답 객체 전체 (choices, usage 등 포함)

```
def call_llm(message: str, temperature: float = 0.7):
    response = client.chat.completions.create(
        model=model_name,
        messages=[{"role": "user", "content": message}],
        temperature=temperature,
    )
    return response
```

Python

```
# 정상 동작 확인
resp = call_llm("안녕? 너는 누구야?")
print(resp.choices[0].message.content)
```

Python



3. Request / Response Model 정의

- 실습 포인트:
 - ChatRequest: 클라이언트로부터 받는 입력 (message)
 - ChatResponse: 클라이언트에 내려줄 출력 (reply, model)
 - 지금은 최소 필드만 정의 — temperature·usage 등은 TODO에서 확장
 - 모델로 받으면 타입·필수 여부 검증을 **FastAPI가 자동으로 해줌**

3. Request / Response Model 정의

/chat 엔드포인트가 받을 입력과 내려줄 출력을 Pydantic 모델로 먼저 정의합니다.

- ChatRequest: 클라이언트로부터 받는 입력 모델
 - message: 사용자 질문 (문자열, 필수)
- ChatResponse: 클라이언트에 내려줄 출력 모델
 - reply: LLM이 생성한 답변 텍스트
 - model: 어떤 모델이 응답했는지

```
from pydantic import BaseModel

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str
    model: str
```

Python



4. /chat 엔드포인트 구현

- 실습 포인트:
 - 흐름: JSON → ChatRequest 자동 검증 → LLM 호출 → 필요한 값만 ChatResponse로 정리해 반환
 - response_model=ChatResponse 지정 → 의도치 않은 필드 노출 방지 + Swagger 문서 자동화
 - LLM 응답에서 우리가 꺼내는 값은 `choices[0].message.content` 와 `response.model` 두 개뿐

4. /chat 엔드포인트 구현

3번에서 정의한 `ChatRequest`, `ChatResponse`를 적용해 `/chat` 엔드포인트를 만듭니다.

흐름:

- 클라이언트가 보낸 JSON → `ChatRequest`로 자동 검증
- `req.message`를 `call_llm`에 넘겨 LLM 호출
- LLM 응답에서 필요한 값만 골라 `ChatResponse`로 정리해서 반환

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest):
    response = call_llm(req.message)
    reply_text = response.choices[0].message.content
    return ChatResponse(reply=reply_text, model=response.model)
```

Python

포인트

- `response_model=ChatResponse`를 지정하면, 함수가 어떤 형태로 반환하든 결국 `ChatResponse` 형태로 정리되어 내려갑니다.
- LLM 응답에서 우리가 필요한 값은 `choices[0].message.content` 와 `response.model` 두 개입니다.
- 아직 오류 처리는 들어가지 않은 상태입니다. 다음 단계에서 붙입니다.



5. 오류 처리 붙이기

- 실습 포인트:
 - LLM API는 외부 서비스 → 언제든지 실패 가능 (Key 만료, Rate limit, 네트워크, 모델명 오류 등)
 - 원칙:
 - LLM 호출 한 줄만 try/except로 감싼다
 - 잡은 오류는 HTTPException으로 변환해 내려준다
 - 원본 예외 메시지(str(e))는 그대로 노출하지 않는다
 - 같은 경로(/chat)에 두 번 등록되어 Duplicate 경고가 뜨는 걸 막기 위해 app = FastAPI()로 라우터를 새로 시작

5. 오류 처리 붙이기

LLM API는 외부 서비스라 언제든지 실패할 수 있습니다.

- API Key 만료, Rate limit, 네트워크 일시 단절, 모델명 오류 등

오류 처리의 기본 원칙:

- LLM 호출 한 줄만 try/except로 감싸기
- 잡은 오류는 HTTPException으로 변환해서 클라이언트에 내려주기
- 원본 예외 메시지를 그대로 노출하지 않기

```
from fastapi import HTTPException

app = FastAPI()

@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest):
    try:
        response = call_llm(req.message)
    except Exception:
        raise HTTPException(status_code=502, detail="LLM 호출 실패")
    reply_text = response.choices[0].message.content
    return ChatResponse(reply=reply_text, model=response.model)
```

Python

포인트

- try 블록 안에는 외부 호출 한 줄만 들어갑니다. 우리 코드의 로직(response.choices[0].message.content 접근 등)은 try 바깥에 둡니다.
- 지금은 모든 실패를 일단 502로 내리지만, 이후 TODO에서 오류 유형별로 상태 코드를 분기합니다.
- detail에는 짧은 카테고리만 넣고, 원본 예외 메시지(str(e))는 절대 그대로 노출하지 않습니다.



6. 서버 실행 및 /docs 테스트

- 실습 포인트:
 - 노트북에서는 uvicorn.run()을 그냥 호출하면 이벤트 루프와 충돌
 - 해결: **별도 스레드**에서 uvicorn 실행
 - Config + Server + threading.Thread(target=server.run, daemon=True).start()
 - 멈추려면 server.should_exit = True 또는 커널 재시작
 - 정상 케이스: /docs → POST /chat → {"message": "..."} → {"reply": "...", "model": "..."}

6. 서버 실행 및 /docs 테스트

작성한 FastAPI 앱을 노트북 안에서 직접 실행합니다.

- `nest_asyncio.apply()` 로 노트북 환경에서도 uvicorn이 실행되도록 합니다.
- 실행 후 출력되는 주소에 `/docs` 를 붙여 접속하면 Swagger UI에서 바로 호출해볼 수 있습니다.
- 서버를 멈추려면 셀의 정지 버튼(■)을 누르세요.

```
import threading
import uvicorn
from uvicorn import Config, Server

config = Config(app=app, host="0.0.0.0", port=8000, log_level="info")
server = Server(config)

thread = threading.Thread(target=server.run, daemon=True)
thread.start()

print("서버 실행 중: http://localhost:8000")
print("문서 페이지: http://localhost:8000/docs")
```

Python

정상 케이스 테스트

`/docs` 페이지에서 `/chat` 엔드포인트를 열고 Try it out → Execute 순으로 호출해보세요.

요청 예시

```
{
  "message": "안녕? 너는 누구야?"
}
```

응답 예시 (값은 실행마다 다름)

```
{
  "reply": "안녕하세요! 저는 ...",
  "model": "openai/gpt-4o-mini"
}
```



7. 실패 케이스 살펴보기

- 실습 포인트:
 - 422: 잘못된 타입 ({`"message": 123`}) → Pydantic이 자동 차단
 - 502: 잘못된 모델명 등 LLM 호출 자체 실패 → 우리가 만든 try/except가 변환
 - 400: 빈 메시지 ({`"message": ""`}) → 지금은 그대로 LLM에 전달됨, TODO 3에서 막을 예정
 - 세 가지 상태 코드가 **각각 다른 역할**을 한다는 점 기억

7. 실패 케이스 살펴보기

`/docs` 또는 curl로 일부러 잘못된 요청을 보내보고, 어떤 응답이 오는지 살펴봅니다.

(1) 빈 메시지

- 요청: `{"message": ""}`
- 결과: 지금은 그대로 LLM에 전달되어 빈 응답이 오거나 어색한 답이 돌아옵니다.
- → TODO 3에서 400으로 막을 예정.

(2) 잘못된 타입

- 요청: `{"message": 123}`
- 결과: FastAPI/Pydantic이 자동으로 **422 Unprocessable Entity**를 반환합니다.
- Request Model 덕분에 우리가 따로 검증 코드를 짜지 않아도 막힌다는 점을 확인하세요.

(3) 잘못된 모델명 (옵션)

- `model_name = "이상한모델"` 처럼 일부러 잘못된 값으로 바꾸고 호출
- 결과: LLM 호출이 실패하면서 우리가 만든 try/except → **502 LLM 호출 실패** 가 떨어집니다.
- 확인 후 `model_name` 은 원래 값으로 되돌려 두세요.



TODO

- TODO 1. Request Model 확장 – temperature 추가
- TODO 2. Response Model 확장 – usage 추가
- TODO 3. 입력 검증 강화 – 빈 메시지 차단
- TODO 4. 오류 처리 분기
- TODO 5. /chat/v2 엔드포인트 추가
- 최종 확인 – 서버 다시 띄우기



최종 확인 – 서버 다시 띄우기

- 실습 포인트:
 - TODO 반영한 app을 새 포트에서 다시 띄워 동작 확인
 - 이전 서버는 `server.should_exit = True`로 정리 또는 포트 변경
 - /docs에서 /chat과 /chat/v2가 모두 정상 동작하는지 확인
- 빈 메시지·범위 밖 temperature·system 유무로 응답 차이 비교

최종 확인 - 서버 다시 띄우기

TODO를 모두 반영한 `app`을 새로운 포트에서 다시 띄워 동작을 확인해보세요.

- 이전 서버가 살아있으면 `server.should_exit = True`로 먼저 멈추거나 포트 번호를 바꿔서 띄웁니다.
- /docs에서 /chat, /chat/v2가 모두 정상 동작하는지 확인하세요.
- 빈 메시지, temperature 범위 밖 값, `system` 프롬프트 있는/없는 경우를 모두 호출해 응답 차이를 비교해보세요.

```
config = Config(app=app, host="0.0.0.0", port=8001, log_level="info")
server = Server(config)

thread = threading.Thread(target=server.run, daemon=True)
thread.start()

print("서버 실행 중: http://localhost:8001")
print("문서 페이지: http://localhost:8001/docs")
```

Python

실습 정리

- 하나의 FastAPI 앱에서 /chat, /chat/v2가 동작
- 입력 검증 → LLM 호출 → 오류 분기 → 응답 정리까지 전체 파이프라인 완성
- 이 형태가 실제 LLM 백엔드에서 가장 기본이 되는 구조입니다.



실습 정리

- 하나의 FastAPI 앱에서 /chat, /chat/v2가 동작
- 입력 검증 → LLM 호출 → 오류 분기 → 응답 정리까지 전체 파이프라인 완성
- 이 형태가 실제 LLM 백엔드에서 **가장 기본이 되는 구조**